

19/5/26 Tue

Java

Why Java is so famous?

Because Banks, companies, and large backend system use Java

* Java was made by Sun microsystem in

(1995) by James Gosling

* Oracle acquired Sun Microsystems in 2010

Since then, Oracle has maintained and controlled the Java platform and made mark

* Java will be updated every 6 months

What is IDE?

→ Integrated Development environment

→ This app helps programmers write, run and

debug code in one place

Examples of IDE:

* Eclipse

* IntelliJ IDEA

* Visual Studio code

Compiler :: translates source code $\rightarrow$ low-level

A compiler is a specialised computer program that translates source code written in a high-level programming language (like Java, C, C++) into a low-level language (like machine code, bytecode, assembly code) so that a computer processor can understand and execute it.

What is JDK?

* Java Development kit is not the compiler itself but it contains the compiler.



Should we need both JRE and Java platform to run a program?

* No, To run a program we only need Java Platform.

* Java platform (JDK/JRE) :: mandatory. Your computer cannot understand or run Java without it.

* JRE :: optional. It is just a smart notepad that makes a programmer's life easier by talking to the Java platform for you.

Why Java files are saved as .java ?

Because - The .java extension tells the compiler :
"Hey the text inside this file is written using the
official Java rules and grammar. Go ahead and
translate it".

⊗ compilation process of Java code :

1) JDK is the overall kit (the toolbox)

2) Inside that kit, Javac is the specific tool
(the compiler)

3) javac takes your human-written .java file
and converts it into machine-friendly bytecode
(.class file)

What if I stored the file name as .java and
created it in C++ hypothetically ?

- Java extension is just the starting point of
the program
"open this file and check its contents as java
source code"

Hello.java

↓
Java compiler (Javac)

↓
opens the file

↓
Read its content

↓
is this valid Java ?

Yes

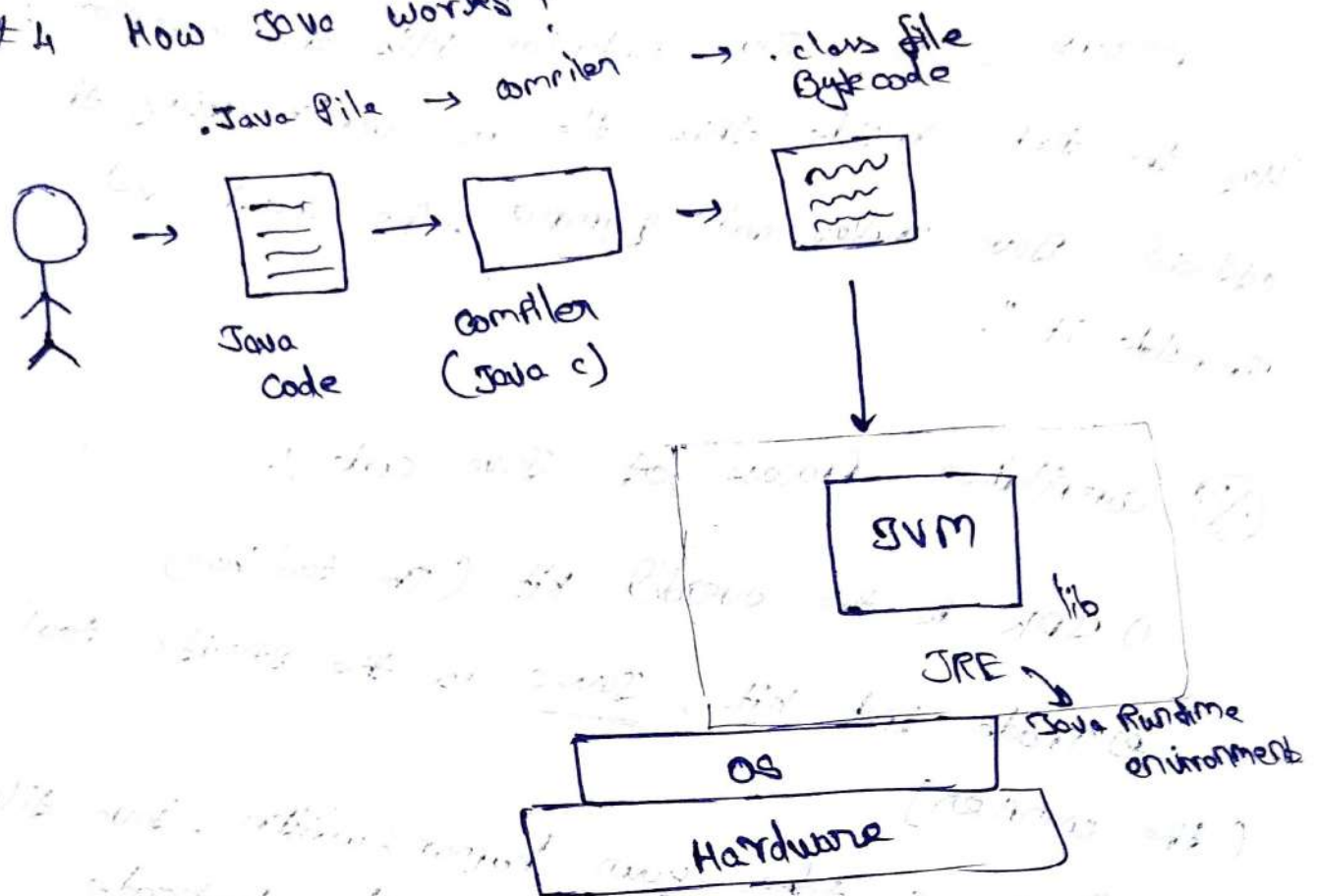
↓
creates
Hello.class

No

↓
compilation error

27/5/26 Wed

#4 How Java works?



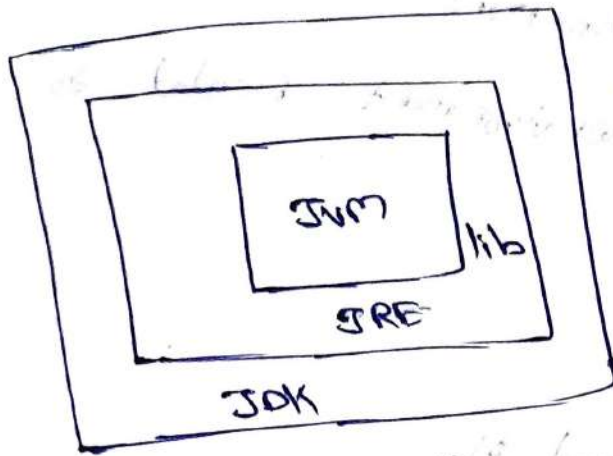
* Java is called Platform independent because the same Java program can run on different OS without changing the code.

* But the JVM is Platform dependent because each OS needs its own JVM implementation.

eg: Windows → JVM for Windows

Linux → JVM for Linux

MacOS → JVM for MacOS



JVM → executes the Program
 JRE → Provides the environment to run the Program
 JDK → Provides everything needed to develop, compile, and run the Program

1) JVM (Java virtual machines)

The JVM is the engine that runs Java Programs

Its main job is:

- * Read bytecode (.class file)
- * Convert bytecode into machine code
- * Execute the Program

Without JVM - Java Programs cannot run

JVM Flow

Bytecode (.class)

↓
JVM

↓
machine code

↓
Program Executes

The JDK compiles the Java source code into bytecode. The JVM, which is part of the JRE, executes the bytecode. During execution, the JVM uses the runtime libraries and supporting files provided by the JRE, whenever needed.

JVM responsibility

- * Executes Java Program
- * converts bytecode into machine code
- * memory management
- * Garbage collection
- * security

2) JRE (Java Runtime Environment)

- * JRE provides the environment needed to run Java Program

It contains:

- + JVM
- + Required lib and files

You can run Java Programs using JRE, but you can't develop/compile Programs

Use of JRE

- * Runs Java application
- * Provides Runtime environment

3) Java development kit:

JDK is used to develop Java Programs

It contains:

- * JRE
- * Compiler
- * debugging tools
- * Packaging tools

⊗ * JDK Provides the tools needed for development (including compilation) and also includes the runtime needed to execute Programs

* JRE Provides the runtime environment

* JVM is the engine that actually executes the code

#5 Variables in Java

Variable - named memory location used to store data

* Variable is generally a temporary storage location for data during Program execution

int age = 20;

int → data type

age → variable name

20 → value stored

Memory - allocated by JVM to store value

6 Datatypes in Java

data type

Primitive
data types

non-Primitive
data types

eg: String - Arrays - classes -
interfaces - Enum

* These are basic built
in data types provided
by Java

* They store actual values
directly

* These are reference types
created by programmers or
provided by Java

* They store references (addresses)
to objects.

Primitive Types in Java:

1 byte = 8 bits

data type	size
1) byte	1 byte
2) short	2 bytes
3) int	4 bytes
4) long	8 bytes
5) float	4 bytes
6) double	8 bytes
7) char	2 bytes
8) boolean	JVM dependent

eg:

byte a = 6;

short b = 100;

int c = 510;

long d = 1000L;

float e = 5.5F;

double f = 99.99;

char g = 'A';

boolean h = true;

7 Literal in Java

Literals are fixed values written directly in Java Program

+ They represent constant values

```
int a = 10;
```

a → variable

10 → literal

Types of literals in Java :-

1) Integer Literals :-

```
int x = 100;
```

Integer Formats

Decimal

Binary

Octal

Hexadecimal

eg: For null

```
String name = null;
```

name is a reference variable

Null means name is not pointing

to any string object

Stack

Heap

name

+ null

(no object exists)

eg:

10

0b1010

012

0xA

Literal Type

1) Integer

2) Floating-Point

3) Character

4) String

5) Boolean

6) null

Example

10

3.14

'A'

"Hello"

true

null

* null is a special literal that means

"This reference variable does not refer to any object"

8 Type conversion in Java

byte b = 127;
int a = 256;

✗ b = a
✓ a = b

automatic →
conversion

byte b = 127;
int a = 12;

b = (byte) a;

←
casting

explicit → ~~conversion~~
casting

float f = 5.6f

int x = (int) f

SOP(x)

O/P: 5 ∴ will cut the point

int a = 257

byte k = (byte) a;

System.out.println(k);

→ byte range
257 % 256

O/P: 1

Type Promotion:

byte a = 10;

byte b = 20;

int c = a + b;

internally Java does:

byte a → int

byte b → int

int + int → int

byte + byte becomes int because
Java does not perform arithmetic
operation directly using byte - short - char

28/5/26 Thu

9 Arithmetic operators in Java

- 1) Assignment operator
- 2) Arithmetic operator
- 3) Relational
- 4) Logical
- 5) Unary
- 6) Bitwise
- 7) Ternary

What is operator?

An operator is a special symbol that tells Java to perform an operation on values or variables.

① Assignment operator:

is an operator used to assign (store) a value into a variable.

Operator

Meaning

Assignment

=

Add and Assign

+=

Sub and Assign

-=

Mult and Assign

*=

divide and Assign

/=

Modulus and Assign

%=

Program

```
int a = 10;
a += 5 // a = a + 5 a = 10 + 5 = 15
System.out.println(a); // 15
a -= 3 // a = a - 3 a = 15 - 3 = 12
System.out.println(a); // 12
a *= 2;
System.out.println(a);
a /= 4;
System.out.println(a);
```

o/p: 15

12

24

6

② Arithmetic operator:

An arithmetic operator in Java is an operator used to perform mathematical calculations on values or variables.

operators

+

operation

Addition

-

Subtraction

*

Multiplication

/

Division

Modulus (remainder)

③ Relational operator:

It is used to compare two values or variables. The result of the relational operation is always

True or False

operator

==

Equal to

!=

Not equal to

>

Greater than

<

Less than

>=

Greater than or equal to

<=

Less than or equal to

Example 1:

```
int a = 10;
```

```
int b = 20;
```

```
System.out.println(a < b)
```

o/p True

Example 2:

```
System.out.println(10 == 9)
```

output

True

④ Logical operator:

* is used to perform logical operation on boolean values (true or false)

operator

meaning

&&

Logical AND

||

Logical OR

!

Logical NOT

1) Logical AND (&&)

+ Returns true only if both conditions are True

```
int age = 20;
```

```
System.out.println(age > 18 && age < 60)
```

o/p:

True

Because both conditions are True:

20 > 18 → True

20 < 60 → True

And:

true && true → true

2) Logical OR (||)

* Return true if at least one condition is true

int age = 70;

Sop(age < 18 || age > 60);

O/P

True

3) Logical NOT (!)

Reverse the boolean value

System.out.println(!true)

O/P:

false

boolean isRain = false;

System.out.println(!isRain);

O/P

True

Short-Circuit Concept in Logical operator

in &&

If the first condition is false,

Java does not check the second condition

eg:

false && anything

Result is false already

in ||

If the first condition is true,

Java skips the second condition

eg:

true || anything

Result is already True.

⑤ Unary operator

* is an operator that works on only one operand (one variable or value)

operator

+

-

++

--

!

~

Meaning

unary plus

unary minus

increment

decrement

Logical NOT

Bitwise complement

1) unary minus (-)

* changes Positive to Negative

int a = 5;

int b = -a;

sof(b)

o/p:

- 5

2) unary plus (+)

int a = 5;

int b = +a;

sof(b)

o/p

5

③ Logical NOT (!)
Reverse boolean value

boolean $x = \text{true}$

System.out.println(!x)

o/p
false

h) Bitwise Complement (~)

* Bitwise complement operator is a unary operator that flips all bits of a binary number

* 1 becomes 0
- 0 becomes 1

eg:-
System.out.println(~5)

o/p - 6

Working:

Binary of 5

0 0 0 0 0 1 0 1

Apply ~

1 1 1 1 1 0 1 0

This represent -6 in 2's complement form

Short cut formula:

$$\sim n = -(n+1)$$

eg:-

$$\sim 5 = -(5+1) = -6$$

5) Increment operator :

(1) Post increment ($a++$)

$+$ uses the value first, then increases it

int $a = 5$;

System.out.print($a++$);

Step 1 :

Current value of a is
5

Step 2 :

$a++$ means

- * use value first
 - * increment later

So 5 is printed

Step 3 :

After printing

a becomes 6

o/p

5

Final value of a :

6

Pre increment ($++a$)

increases the value first, then uses it

~~Step 1~~

int $c = 5$;

SOP($++c$);

Step 1 :

Current value of c is 5

$c = 5$

Step 2

$++c$ means

- * increment first
 - * uses later

So :

c becomes 6

Step 3

Now 6 is printed

o/p

6

Final value of c :

6

6) Decrement operator :

Similarly for post increment and decrement.

⑥ Binary operators in Java :-

* are operators that perform operations directly on the binary representation of integers.

Operator	Name	Working
&	Binary AND	1 only if both bits are 1
	Binary OR	1 if atleast one bit is 1
^	Binary XOR	1 if bits are different
~	Binary complement	Flips all bits
<<	Left shift	Shifts bits left
>>	Right shift	Shifts bits right
>>>	Unsigned right shift	Right shift with zero fill

Left Shift :-

The left shift operator shifts all binary bits of a number towards the left side.

Syntax

number << Position

eg:-

5 << 1

Meaning:-

Shift all bits of 5 left by 1 positions

$$5 << 1$$

$$5 = 00000101$$

Shift left by 1

$$00001010$$

convert back to decimal

$$1 \times 2^3 + 0 \times 2^2 = 8 + 0 = 8$$

So:

$$5 << 1 = 10$$

$$5 \times 2^1 = 10$$

Formula

$$N << K = N \times 2^K$$

where:

$N \rightarrow$ Number

$K \rightarrow$ Number of shifts

Right shift:

Number $\gg$ Positions

$$8 \gg 1$$

Shift all bits of 8 right by 1 positions

$$8 = 00001000$$

Shift by 1

$$00000100$$

convert back to decimal

So

$$8 \gg 1 = 4$$

Formula

$$N \gg K = \frac{N}{2^K}$$

where:

$N \rightarrow$ Number

$K \rightarrow$ Number of shifts

⑧ Ternary operator:

It is a conditional operator that returns one value if the condition is True and another value if the condition is false

```
int a = 10;
```

```
int b = 20;
```

```
int max = (a > b) ? a : b
```

Syntax

condition ? value 1 : value 2

* If condition is true → return value 1

* Else → return value 2

29/5/26

Types of conditional statements in Java

- 1) if
- 2) if-else
- 3) else-if ladder
- 4) nested if
- 5) switch

1) if statement

* used to execute code only when the condition is true

Syntax

```
if (condition)
```

```
{
```

```
    //code
```

```
}
```

true → code executes

false → skipped

2) if-else statement

* used when there are two choices

```
if (condition)
```

```
{  
    // true block
```

```
}  
else
```

```
{  
    // false block
```

```
}
```

3) else if Ladder :

* used to check multiple conditions

```
if (condition 1)
```

```
{  
    // code
```

```
}
```

```
else if (condition 2)
```

```
{  
    // code
```

```
}
```

```
else if (condition 3)
```

```
{  
    // code
```

```
}
```

```
else
```

```
{  
    // code
```

```
}
```

4) Nested if

* An if statement inside another if

Syntax

```
if (condition 1)
```

```
{  
    if (condition 2)
```

```
{
```

```
}
```

```
}
```

```

9. int age = 20;
   boolean hasID = true;
   if (age >= 18)

```

```

       if (hasID)
           System.out.println("Entry allowed");
   }
}

```

5) Switch Statement - used to select one option from many ideas

Example:

```
int day = 2;
```

```
switch (day)
```

```
{
```

```
    case 1:
```

```
        System.out.println("Monday");
```

```
        break;
```

```
    case 2:
```

```
        System.out.println("Tuesday");
```

```
        break;
```

```
    default:
```

```
        System.out.println("Invalid");
```

```
}
```

o/p

Tuesday

* The default case executes when none of the case match switch expression

* If there is a break statement it is not used. Java executes the next case even after finding a match

Need of loops in Java

+ loops are used to execute a block of code repeatedly

+ instead of writing the same statements again and again, we use loops

Type

+ while

+ do while

+ for

while :

A while loop is used to repeatedly execute a block of code as long as the condition is true

Syntax

while (condition)

{
 //code
}

Program

1 // Java program to print numbers from 1 to 5

2 int i = 1;

3 while (i <= 5)

{
4 System.out.println(i);

5 i++;

6 }

Output

1
2
3
4
5

Where while loop is used in Java

- 1) When number of iteration is unknown
- 2) When loop depends on condition
- 3) When Repeating tasks continuously

do with

* The code must decrease at least one time before checking the condition

Screen 2 - New Screen (system.in) -
int Password:

```
do
    system and print ("Enter Password:");
    Password = sc.next();
while (Password != 1234);
sop ("Login successful");
```

Q/c Enter Password : 1111
Enter Password : 1234
Login Successful

For loop : for loop is used to repeat a block of code a fixed number of times.

↓ initialization condition increment

for (int i = 1; i <= 5; i++)

initialization condition increment
do { int i = 1 ; i <= 5 ; i++ }

System.out.println(i);

2

5/5

5321

20/3/20 Sat

#21 class and object Theory in Java

What is class and object?

Let's understand with example:

Imagine a student:

Suppose you have 2 students

Hari

John

Each student has

• Name

• Age

• Marks

Step 1: Create a class

A class is a blueprint / template that describes what a student should have

class Student

{

String name; // instance variable

int age; // instance variable

int marks; // instance variable

}

This class says:

Every student object will have a name, age, and marks

But at this stage no student exist yet

Step 2: Create objects

Now create actual Students variable - refer to object created in memory

class name Students $S1 = \text{new Student}()$ creates a new object in heap memory
class name Students $S2 = \text{new Student}()$

Here:

$S1$ is an object

$S2$ is ~~an object~~ another object

Step 3: Store data

$S1.name = "Hori"$

$S1.age = 20$

$S1.marks = 95$

$S2.name = "John"$

$S2.age = 21$

$S2.marks = 88$

memory looks like

(Heap)

$S1$

name = Hori

age = 20

marks = 95

$S2$

name = John

age = 21

marks = 88



* Data are created from the same class

* Each object store different data



What is class?

A class is a blueprint or template used to create objects

What is object?

An object is a real instance of class that

contains actual data



So,

A class is a blueprint, and an object is the real thing created from that blueprint

1/6/20 Mon

What is Methods?

A method is a block of code that performs a specific task. Instead of writing the same code again and again, you can put it inside a method and call it whenever needed.

4 Types of Methods

1) No Parameter, No Return Value

```
public class main {
```

```
    static void greet () {
```

```
        System.out.println("Hello");
```

```
    }
```

```
    PSUM 4  
    greet () :- // method
```

2) Parameter, No Return Value

```
public class main {
```

```
    static void greet (String name) {
```

```
        System.out.println("Hello" + name);
```

```
    }
```

```
    PSUM 4  
    greet ("Hori");
```

When to use static?

"Does this method need to use data from a particular object?"

No - make it static (like your greet() method)

Yes - non static

3 No Parameter - Return value

```
Public class main {
```

```
    static int getNumber () {
```

```
        return 100;
```

```
    }  
    public static void main (String args) {
```

```
        int num = getNumber ();
```

```
        System.out.println (num);  
    }  
}
```

4) Parameters and Return value

```
Public class main {
```

```
    static int add (int a, int b)
```

```
    {  
        return a+b;
```

```
    }  
    public static void main (String [] args) {
```

```
        int result = add (10, 20);
```

```
        System.out.println (result);  
    }  
}
```

What is method overloading?

Method overloading is a feature in Java where multiple methods have the same name but different parameter lists (different number, type or order of parameters).

Uses:

It improves code readability, reusability and supports

Compile-time Polymorphism

static class main {
 static int add (int a, int b) {
 return a+b;
 }
}

static int add (int a, int b, int c) {
 return a+b+c;
}

PSUM 1

System.out.println (add (10, 20));

System.out.println (add (10, 20, 30));

3

2/4/2020 Tue

What is Stack memory?

Stack memory is the memory area used to

store

- local variable
- method parameters
- method calls
- Reference variable (addresses of objects)

9. public class main {

PSUM 1

int a = 10;

int b = 20;

3 3

Here a and b are local variables. So they are stored in stack

Stack

Main ()

a = 10

Rules (Type of method parameters)
 1) No of parameters
 add (int a, int b)
 add (int a, int b, int c)
 2) Type of parameters
 add (int a, int b)
 add (double a, double b)
 3) Order of parameters
 Print (int a, String b)
 Print (String b, int a)

Method call example:

```
public class Demo {
```

```
    PSUM {
```

```
        add(3, 4)
```

```
    }
```

```
    static void add(int x, int y) {
```

```
        int sum = x + y;
```

new stack frame is created for add
After add() finishes, its stack frame is removed.

```
add()
```

```
x = 3
```

```
y = 4
```

```
sum = 7
```

```
Stack
```

```
main()
```

Each stack frame is removed after its method finishes executing. When the method (main) finishes, the entire stack becomes empty.

↓
LIFO - Last method called is the first one to finish or be removed.

Characteristics of stack memory:

- * stores method-related data
- * store local variables and references
- * very fast access
- * follows LIFO

⊗ Each method call gets its own stack frame

What is Heap Memory in Java?

Heap memory is the memory area where Java

stores

- * Objects

- * Arrays

- * instance variable

Whenever you use the new keyword an object is created in the heap

Example

class Calculator {

int num = 5; // instance variable

}

public class Demo {

PSVM {

Calculator obj = new Calculator();

}

}

Memory

Stack

obj → object

Heap

Calculator object

num = 5

* object (reference variable) is stored in the stack

* the actual Calculator object is stored in the heap

* num is an instance variable so it is stored in heap

Local variable

* A local variable is a variable declared inside a method, constructor or block

eg: class Demo {

void show() {

int a = 10; // local variable

sort(a);

}

instance variable

* is a variable declared inside a class but outside all methods

eg: class Student {

String name = "Harri";

void display() {

int marks = 90; // local variable

sortName(name);

sortName(marks);

}

}

Characteristics

- * Declared inside a method
- * Stored in a stack memory
- * Created when method starts
- * Destroyed when method ends
- * Accessible only inside that method
- * No default value

Memory

Heap (Object)

Student object

Name = "Kori"

Stack (display method)

display()

marks : 90

- * Declared inside a class - outside methods
- * Stored in Heap memory
- * Each object gets its own copy
- * Exist as long as the object exists
- * Accessible through the object
- * Gets default values (0, null, etc.)

Why Java has two types of memory?

Java uses stack and heap memory to efficiently manage temporary method data and long lived objects. Stack stores local variables and method calls for fast access, while heap stores objects and instance variables that may be used throughout the program.

Local Variable

Local variable exists only inside the methods where it is declared.

Instance Variable :- Instance variable can be accessed by creating many objects.

What is the Need of an Array?

- * Array is used to store multiple values of the same data type in a single variable
- * It reduces the need for creating many variables
- * It makes data processing easier using loops
- * It helps in storing and accessing data efficiently using index

Ex:

Without Array

int a = 10;

int b = 20;

int c = 30;

With Array

int[] arr = {10, 20, 30};

Array Creation:

int[] arr = new int[5];

int → data type

arr → array name

new → create the array

5 → size of the array

9/6/26 Wed

1 Dimensional Array

```
import java.util.Scanner;
```

```
public class array {
```

```
    public static void main (
```

```
        System.out.println ("Enter n:");
```

```
        Scanner in = new Scanner (System.in);
```

```
        int n = in.nextInt();
```

```
        int arr[] = new int [n]
```

```
        for (int i=0; i < n; i++) {
```

```
            arr[i] = in.nextInt();
```

```
        }
```

memory diagram

9/6 Enter n : 5

1 2 3 4 5

Stack

main()

n=5

in

arr

Heap

Scanner object

Array object

[0] [1] [2] [3] [4]

2D Array

```
int arr [][] = new int [m][n]
```

3D Array

```
int arr [][][] = new int [m][n][k]
```

Jagged Array:

Each row can have a different number of elements

```
int nums[][] = new int[3][]
```

```
nums[0] = new int[3];
```

```
nums[1] = new int[4];
```

```
nums[2] = new int[2];
```

Drawback of Array:

1) Fixed size

2) Wastage of memory

3) Can store only same type of data

4) Insertion of element takes time

5) Deletion of element takes time

6) No built in dynamic operation

Array of objects:

The main use of an array of objects is to store multiple objects of the same type together and process them easily using loops.

Class Student {

int rollno;

String name;

int marks;

}

Public class demo {

2 public static void main (String args) {

Student s1 = new Student ();

s1.rollno = 1;

s1.name = "Ravi";

s1.marks = 88;

Student s2 = new Student ();

s2.rollno = 2;

s2.name = "Rohit";

s2.marks = 67;

System.out.println (s1.name + " : " + s1.marks);

creating
an array
↑

⊗ Student student [] = new Student [3];

Student [0] = s1;

Student [1] = s2;

Student [2] = s3;

} does not create new objects

It only copies reference

for (int i = 0; i < student.length; i++) {

1

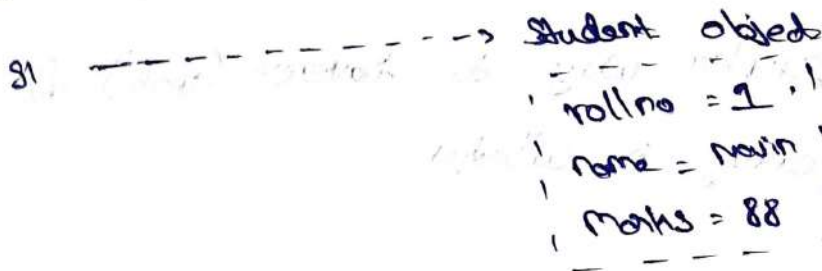
System.out.println (student [i].name + " : " + student [i].marks);

2

Before creating the array:

Stack

Heap

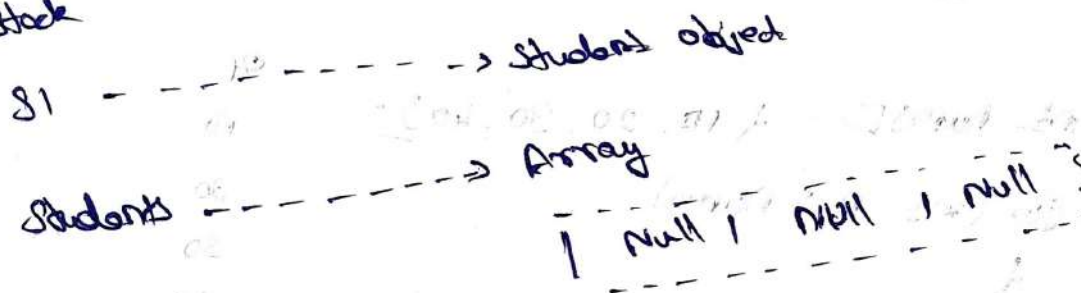


After creating the Array:

Student Student[] = new Student[3]

Stack

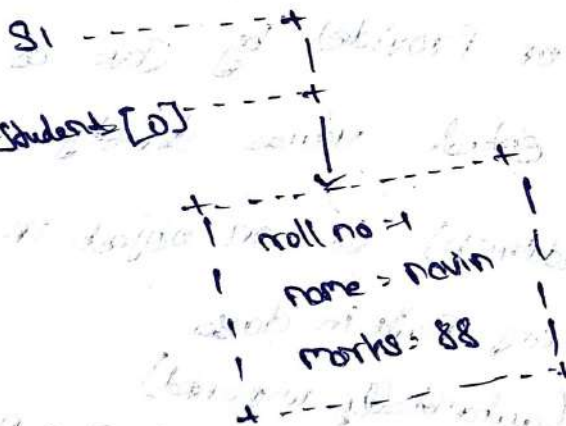
Heap



Store reference

Student[0] = S1

Student[1] = S2



S1 and Student[0] points to the same object

S2 and Student[1] points to the same object

* copying a reference means copying the object's address from one reference variable to another. No new object is created. Both references point to the same object.

4/6/26 Thu

Enhanced for loop :- (or) For-each loop

It is a simpler way to traverse (visit) all elements of an array or collection

Syntax

```
for (data type variable : array)
```

```
{
```

```
}
```

eg:

```
int nums[] = {10, 20, 30, 40};
```

```
for (int n : nums)
```

```
{
```

```
    solve(n);
```

```
}
```

of

10

20

30

40

50

What is non-primitive data type?

Non-primitive data types are data types that are created by the programmer or provided by Java as classes. They do not store the actual values directly; instead, they store a reference (address) to an object in memory.

Packages containing Built-in classes

eg: String

Arrays

Classes

Objects

Interfaces

java.lang (Automatically imported)

* String * Date * System * Integer * Double

java.util

* Scanner * Arrays * ArrayList * HashMap * Vector

java.io

* File handling classes

eg:

String name = "Hari";

* name is a reference to a String object

int nums[] = {10, 20, 30};

* nums is a reference to an array object

Student s1 = new Student();

* s1 is a reference to an Student object

Non-Primitive Types (created using classes)

Java provides some classes:

String name = "Hari";

Here String is a class Provided by Java

⊗ You can also create your own classes:

class Student {

int roll no;

String name;

Student s1 = new Student();

* Student is a non-primitive type because you

created it

your class

Student

Student s1 = new Student();

created by you

Java class

String

String name = "Hari";

created by Java

Why does string looks different?

Normally, objects are created like:

Student s1 = new Student()

But for convenience - Java allows:

String name = "Hari"

instead of always writing

String name = new String("Hari")

Memory View

Stack

Heap

name -----> String Object ("Hari")

s1 -----> Student Object



* String is a predefined class in Java

* Student is a user-defined class in Java

* Both create objects and the variable (name, s1)

store reference to these objects.

What is string constant pool (SCP)?

The string constant pool is a special area in memory where java stores string literals to some memory.

eg:

String S1 = "Hari";

String S2 = "Hari";

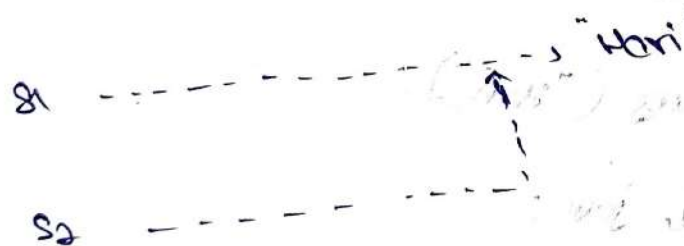
When Java sees "Hari" for the first time:

- 1) It checks the string constant pool
- 2) If "Hari" is not present, it creates it
- 3) S1 points to that string object.

When Java sees "Hari" again:

- 1) It checks the pool
- 2) "Hari" already exists
- 3) S2 points to the same object

Stack



⊗ only one object "Hari" is created

Advantages of SCP:

* Saves memory

* Improves performance

* What about new String () ?

String s1 = "Hori";

String s2 = new String ("Hori");

Memory:

String constant Pool

"Hori"

Heap

String Object ("Hori")

Here:

s1 points to the Pooled String

s2 points to a new object in heap

So they are different objects

* Why does this happen

because new always means "create a new object" in Java

String s2 = new String ("Hori");

The keyword new tell java:

"I don't care whether an object already exists
create a fresh object for me"

* What is the use of this ?

* Every classes in Java can create object with new so String also does this

* Sometimes you may want a completely independent object

Stack

s1 -----> SCP "Hori"

s2 -----> Heap String

In Java 7 and later
String constant pool is
stored in heap memory
Before Java 7, it
was stored in Perm Gen
area of the JVM
Area.

String is immutable in Java. Once a String object is created, its content can't be changed. Operations like `concat()` create a new String object instead of modifying the existing one.

eg:
String name = "Hari";
name = name.concat("Shankar");

Now:

- 1) "Hari" exists
- 2) `concat()` creates a new string "Hari Shankar"
- 3) Name is updated to point to the new string

Before:

name -----> "Hari"

After

name -----> "Hari Shankar"

"Hari" still exists until garbage collected

~~why~~ why are strings immutable?

Security reasons

- * Strings are used for Password, File Paths, URL, etc.
- * Their values can't be changed unexpectedly.

* What is StringBuffer in Java?
StringBuffer is a class used to create mutable string

* What is capacity?
Capacity is the total number of character a StringBuffer can hold before increasing its memory

StringBuffer sb = new StringBuffer();

Length = 0

Capacity = 16

Length vs capacity

StringBuffer sb = new StringBuffer("Hari");

Length = 4 (Hari)

Capacity = 20 (4 + 16)

* Why extra 16 spaces?

Java reserves 16 extra spaces for future append() operation

[H] [a] [r] [i] - - - - -
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

sb.append("Shrikant");

Java uses the free spaces already available in the buffer

No new allocation is needed until the capacity becomes full

When capacity becomes full

Java automatically increases capacity using

new capacity = (old capacity $\times$ 2) + 2

eg: old = 20

* What is StringBuilder in Java?

- * StringBuilder is a class used to create mutable string
- * Just like StringBuffer, it allows you to modify the same string object without creating new object repeatedly

String → immutable
1) StringBuffer → mutable, thread safe
2) StringBuilder → mutable, not thread safe

What is Static Variable in Java?

A static variable is a ~~static~~ variable that belongs to the class, not to object.

- * only one copy is created

- * All objects share the same copy

Static variable is not stored inside an object.

It is stored in a special memory area associated with the class, often called the class area (or method area in JVM terminology).

Program

```
class Student
```

```
{  
    String name; // instance variable
```

```
    static String college; // static variable
```

```
    void display ()
```

```
{  
        System.out.println("name : " + name)
```

```
        System.out.println("college : " + college)
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
        Student S1 = new Student()
```

```
        S1.name = "Hani"
```

```
        Student.college = "VCE"
```

```
        Student S2 = new Student()
```

```
        S2.name = "Raj"
```

```
        Student.college = "VCE"
```

```
        Student.college = "VCE"
```

```
        S1.display()
```

```
        S2.display()
```

```
}
```

9/6/26 Tue

Understanding JDK, JRE, JVM, Stack, Heap, classes, objects

Program:

class Hello {

public static void main (String args [])

{
System.out.println ("Hello world");

}

Step 1: Write the Program

you write the code in:

Hello.java

This is called Source code

Step 2: Compilation (JDK)

When you run:

javac Hello.java

The java compiler (javac) converts the source code into
bytecode

Hello.java (source code)

↓
javac

↓
Hello.class (byte code)

Bytecode is language understood by JVM

Step 3: Execution starts

When you run:

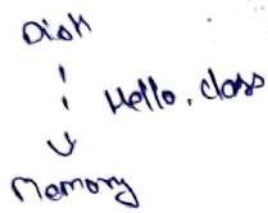
java Hello

The JVM starts

java Hello

↓
JVM

Step 4: Class Loading
 The JVM asks the class loader to load
 Hello.class
 into memory.



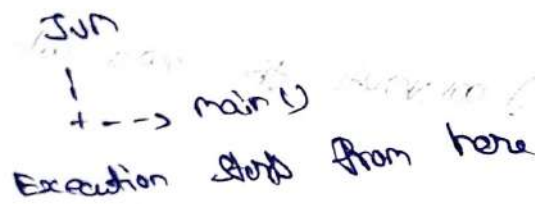
Now JVM knows about the Hello class

Step 5: Find main()

JVM searches for

Public static void main(String args[])

because it is the entry point of every Java application

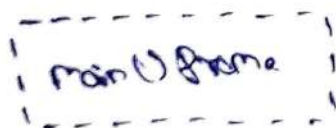


Step 6: Create Stack Frame

Whenever a method is called, JVM creates a
 Stack Frame

For main():

Stack



Step 7: Execute Statement

JVM reaches:

System.out.println("Hello world");

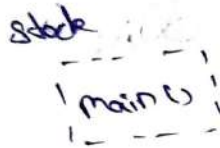
Output

Hello world

Step 8: main() ends

The stack frame is removed

Before:



After

stack:

(empty)

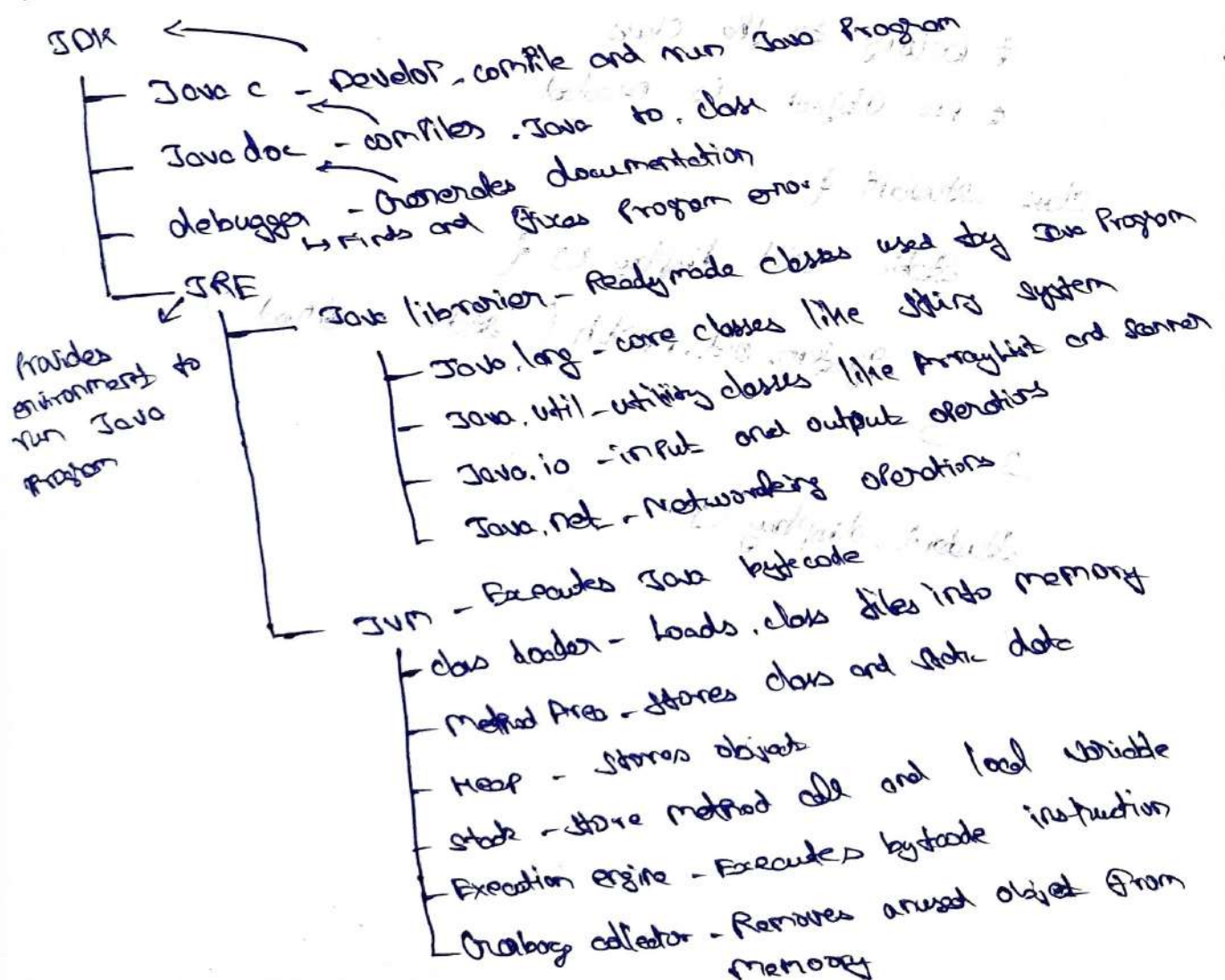
Step 9: JVM terminates

Since there are no more statements to execute

Program ends

JVM shuts down

Complete Hierarchy:



10/6/26 Thru

What is Non-Static Method?

+ Belongs to an object.

+ you must create an object before calling it.

Program :

```
class Student {
```

```
    void display () {
```

```
        System.out.println ("Non-Static Method");
```

```
    }  
    Student s = new Student ();
```

```
    s.display ();  
}
```

What is Static Method?

+ Belong to the class

+ No Object is needed

```
class Student {
```

```
    static void display () {
```

```
        System.out.println ("Static Method");
```

```
    }  
    Student.display ();  
}
```

In "Public static void main (String args [])" what is the use of static?

The static keyword is used so that the JVM can call main() without creating an object of the class.

Imagine if main() was not static.

class Demo {

public void main (String args []) {
 System.out.println ("Hello");
}

3 3

When the program starts, the JVM needs to execute main(). But to call a non-static method, it must first create an object.

Demo d = new Demo ();

d.main (args);

But the problem is: who creates this object?

The JVM would need an object before it can call main(), but main() itself is supposed to be the starting point of the program.

Execution Flow:

You run: java Demo

↓
JVM loads Demo class

↓
JVM finds main()

↓
JVM calls Demo.main (args) (internally calls)

↓
Program starts executing

What is Static Block?

class demo {

static {
System.out.println("Static Block Executed");

}
public static void main (String [] args) {
System.out.println("main method executed");

}

O/P

Static Block Executed

main method executed

Execution :

Run Program

↓

class loaded

↓

Static Block executes

↓

main() executes

* The JVM first loads the Demo class. During class loading, it executes the static block. After that it calls the main() method.

4 Pillars of object oriented Programming (OOPS)

1) Encapsulation

2) Inheritance

3) Polymorphism

4) Abstraction

What is encapsulation?

Encapsulation is the process of hiding data using private variables and providing controlled access through public getter and setter methods.

Program

class BankAccount {

private double balance;

public void setBalance(double amount) {

if (amount >= 0) {

balance = amount;

}

public double getBalance() {

return balance;

}

public class Demo {

public static void main(String[] args) {

BankAccount acc = new BankAccount();

acc.setBalance(5000);

System.out.println("Balance = " + acc.getBalance());

}

o/p

Balance = 5000.0

Notes version:

Getter:

get Balance()

→ Read Balance

Setter:

set Balance

→ Modify Balance

Why?

Without encapsulation:

acc. balance = -10000

With encapsulation:

only valid values can be stored

Advantages:

- 1) Data Hiding
- 2) Security
- 3) Controlled Access
- 4) Better maintainability

8/7/26

Getter:

A getter is a Public Method used to read (get) the value of Private instance variable

Setter:

A setter is a Public Method used to update (set) the value of Private instance variable

This Keyword

+ this refers to the current object

* It is used to access the current object's instance variable and methods

+ It helps distinguish an instance variable from a method parameter when they have the same name.

class Human {

private int age;

public void setAge(int age) {

this.age = age;

public int getAge() {

return age;

public class Demo {

public static void main (String[] args) {

Human obj = new Human();

obj.setAge(30);

System.out.println(obj.getAge());

How

initially:

object (Heap)

age = 0

when you call:

obj.setAge(30);

The method parameter becomes
parameter (stack)

age = 30

How

this.age = age

Java understands as

object's age = parameter
age

So -

Before

object age = 0

parameter age = 30

After

object age = 30

parameter age = 30

* The old value 0 is replaced
with new value (30)